

Monitoring





Prometheus and SLO Dashboards

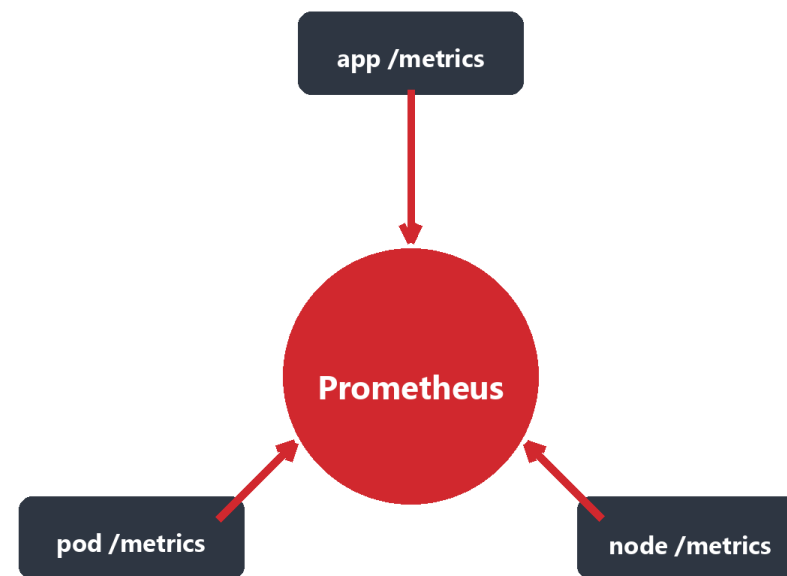
- Prometheus is at the center of how you reason about reliability. You will collect real metrics and turn them into the availability and latency numbers an SLO is actually built from.
 - Time series database
 - Pull based collection
 - Metrics focused
 - SLO driven



What Prometheus Is

- Prometheus is the open-source standard for collecting and storing metrics in cloud-native systems. It scrapes numbers from your services every few seconds and keeps them as time series you can query, alert on, and graph.
 - Metrics database
 - Scrapes endpoints
 - Label based model
 - Kubernetes native

■ What Prometheus is

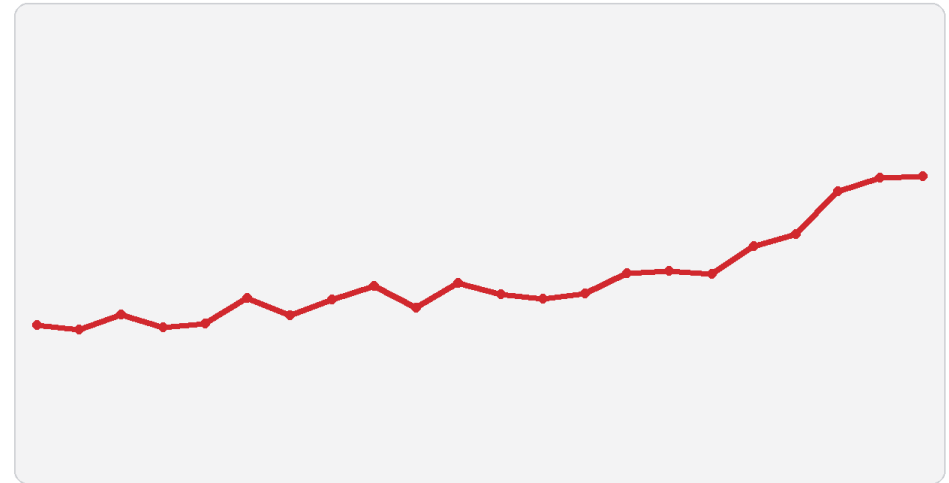


Scrapes targets on a schedule

Metrics as Telemetry

- Metrics are the cheap, always-on signal a system emits about itself. A handful of counters and gauges, sampled continuously, is usually enough to see a problem forming before users do.
 - Counts
 - Rates
 - Latencies
 - Resource usage

■ Metrics as telemetry



Cheap, continuous, always on

Prometheus Data Model

- Prometheus does not just store a number, it stores a number plus the labels that describe it.
http_requests_total with handler and status labels is one series, and those labels are what let you slice traffic by route or status later.
 - Metric name
 - Key value labels
 - Timestamped values
 - Watch label cardinality

■ Metric name + labels = one series

```
http_requests_total{  
  handler="/checkout",  
  status="500" }    = 1420
```

Metric Types

- Prometheus gives you a few metric types, and picking the right one matters. A counter that only climbs, a gauge that moves both ways, and a histogram of durations each answer a different question.
 - Counter
 - Gauge
 - Histogram
 - Summary

Metric types

Counter

- only goes up

Gauge

- up and down

Histogram

- buckets

Summary

- quantiles

Counters

- A counter only ever goes up, which makes it perfect for counting events like requests or errors. You rarely read the raw value; you wrap it in `rate()` to ask how fast it is climbing right now.
 - Request counts
 - Error counts
 - Always increasing
 - Use `rate()`

Counters only go up

```
http_requests_total 1,420,388  
  
rate(http_requests_total[5m])  
= 240 req/s
```


Gauges

- A gauge is a value that moves up and down, a snapshot of right now. Memory in use, in-flight requests, and queue depth are gauges, and you read them directly rather than as a rate.
 - In progress requests
 - Memory usage
 - CPU usage
 - Queue size

■ Gauges: a value, up or down

✓ Memory in use

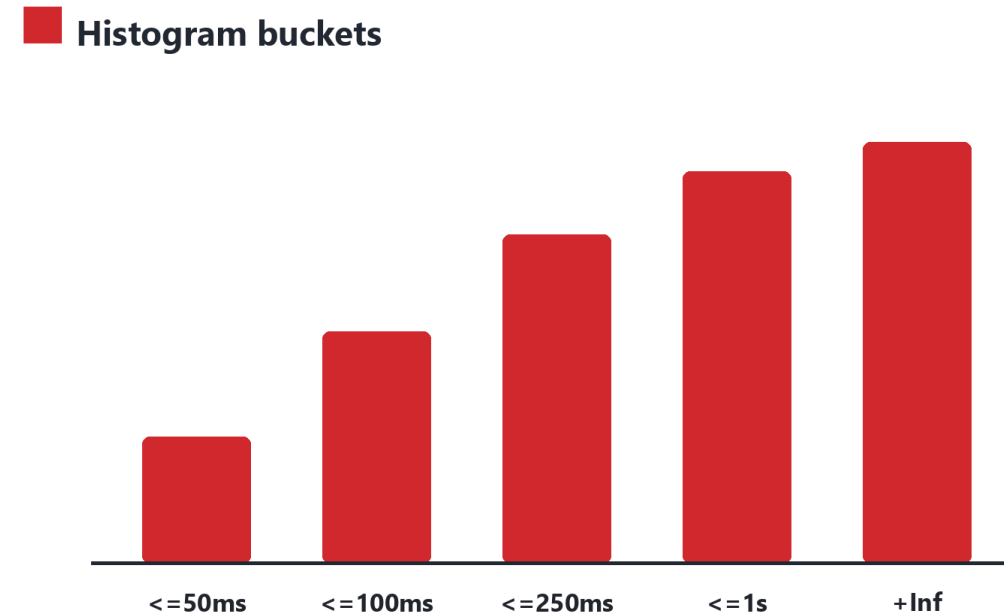
✓ In-flight requests

✓ CPU usage

✓ Queue depth

Histograms

- A histogram sorts observations into buckets, which is how Prometheus tracks latency without storing every request. From those buckets you compute percentiles with `histogram_quantile`, so you can ask how slow the 95th percentile request is.
 - Latency buckets
 - Request duration
 - Percentiles
 - `histogram_quantile`



`histogram_quantile -> p95`

Time Series

- Every metric is a stream of timestamped points, one series per unique set of labels. PromQL always operates over these series, which is why thinking in time series is the key to reading any dashboard.
 - Timestamped points
 - Continuous data
 - Historical trends
 - Graphable

Time series

```
cpu{instance="a"}    0.42  0.55  0.61
cpu{instance="b"}    0.30  0.34  0.29
```

one series per label set

PromQL Purpose

- PromQL is how you turn raw metrics into answers. Filtering, aggregating, and computing rates is what takes you from we collect `http_requests_total` to error rate on checkout is 2 percent and rising.
 - Filter metrics
 - Aggregate values
 - Calculate rates
 - Build SLOs

■ PromQL turns metrics into answers

```
rate(http_requests_total[5m])  
sum by (handler) (...)  
errors / total = SLO
```

Instant vs Range Vectors

- PromQL has two shapes of data, and knowing which one you have prevents most beginner errors. An instant vector is the value right now; a range vector is a window of recent values that functions like `rate()` need.
 - Instant vector
 - Range vector
 - Single timestamp
 - Time window

Instant vs range vectors

Instant vector

- value right now
- one point per series
- used for gauges

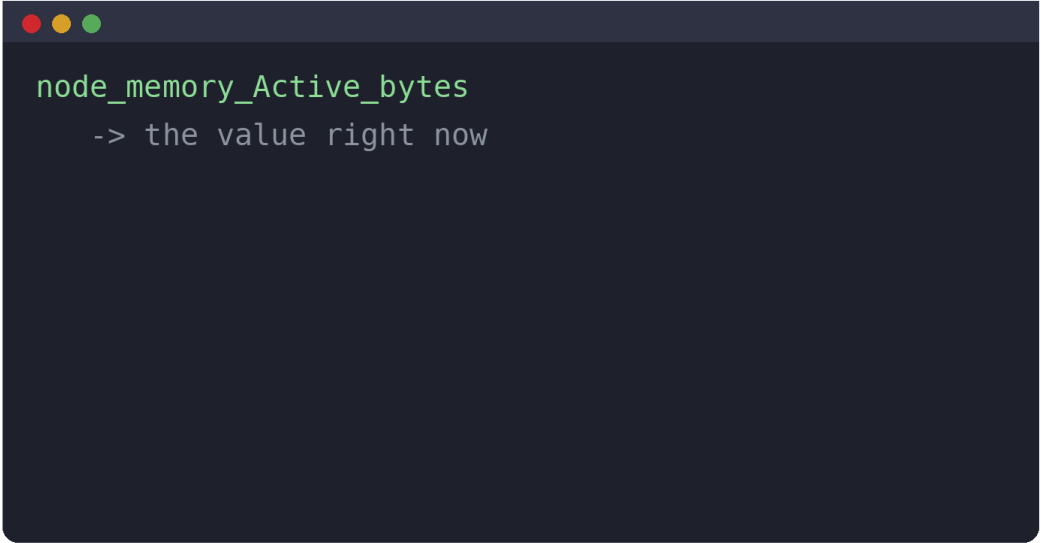
Range vector

- a window, e.g. [5m]
- many points
- needed by `rate()`

Instant Vectors

- An instant vector is the latest value of each series, the answer to what is happening right now. It is what you graph for a gauge or read at the top of a dashboard.
 - Current state
 - Used for gauges
 - Point in time
 - Live view

Instant vector



```
node_memory_Active_bytes  
-> the value right now
```

Range Vectors

- A range vector is a series sampled over a window, written with a duration like [5m]. You need it whenever you ask about change over time, because a single point cannot tell you a rate.
 - Time window
 - Used for counters
 - Historical view
 - Trend analysis

Range vector

```
http_requests_total[5m]  
-> points over 5 minutes
```


Why rate() Exists

- A raw counter like 1,420,388 means nothing on its own. `rate()` turns that ever-climbing number into a per-second value, so requests per second and errors per second become things you can graph and alert on.
 - Events per second
 - Normalizes counters
 - Used for traffic
 - Used for errors

■ `rate()` makes counters useful

```
counter: 1,420,388 (meaningless)

rate(http_requests_total[5m])
= 240 req/s
```

Aggregation Operators

- A busy service emits one series per pod, which is too much detail to reason about. sum, avg, max, and min collapse many series into one, so you can think about the service instead of each instance.
 - sum
 - avg
 - max
 - min

Aggregation operators

sum

avg

max

min

Reason about the service, not each pod

The sum by Pattern

- sum by is the PromQL pattern you will use most. sum by (handler) rolls hundreds of per-pod series up into one line per route, which is exactly how a dashboard panel is built.
 - Group by label
 - Collapse dimensions
 - Service level view
 - Cluster view

■ sum by: one line per route

```
sum by (handler) (  
    rate(http_requests_total[5m])  
)
```

Ratio Queries

- Most reliability numbers are really a ratio of two metrics. Dividing successful requests by total requests gives availability, and that single division is the math behind almost every SLO.
 - Error rate
 - Availability
 - Success percentage
 - Reliability math

■ Ratios power SLOs

```
sum(rate(success_total[5m]))  
/  
sum(rate(requests_total[5m]))  
= availability
```

Availability Mental Model

- Availability is simply the share of requests that succeed, measured from the user's side. Framing it as good requests over total keeps the focus on whether people can actually use the system, not whether a server is pingable.
 - Success divided by total
 - User focused
 - Uptime metric
 - SLO foundation

■ Availability

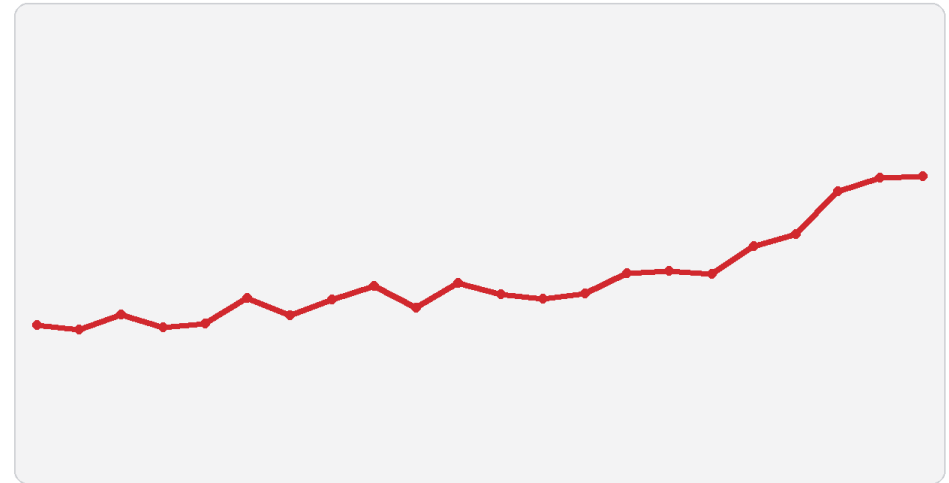


Measured from the user side

Latency Mental Model

- Latency is how long requests take, and it is the clearest proxy for how fast the system feels. Because a few slow requests matter more than the typical one, latency is best read as a distribution, not a single number.
 - Response time
 - Performance signal
 - Distribution based
 - Not averages

■ Latency over time

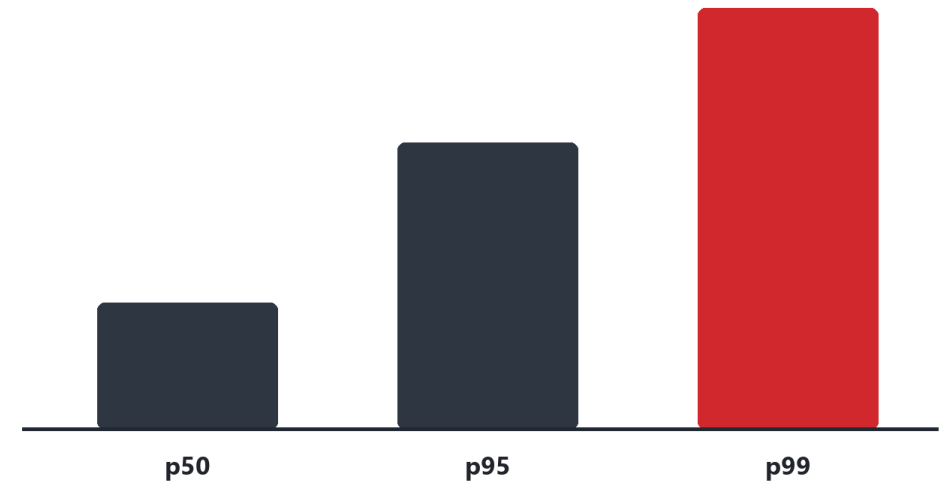


Read it as a distribution

Why Percentiles Matter

- An average latency of 200ms can hide the fact that 1 in 20 users waits 3 seconds. Percentiles like p95 and p99 surface that tail, which is the experience that drives complaints and churn.
 - P50
 - P95
 - P99
 - Tail latency

■ Percentiles show the tail



The average hides the slow 1%

MCQ

- What type of database is Prometheus?
 - Relational database
 - Document database
 - Time series database
 - Log database



Answer

- What type of database is Prometheus?
 - Relational database
 - Document database
 - **Time series database**
 - Log database



MCQ

- Which metric type should be used to count requests?
 - Gauge
 - Counter
 - Histogram
 - Summary



Answer

- Which metric type should be used to count requests?
 - Gauge
 - **Counter**
 - Histogram
 - Summary



MCQ

- Why is rate() commonly used with counters?
 - To reset counters
 - To convert counters into per-second signals
 - To delete old data
 - To group labels



Answer

- Why is rate() commonly used with counters?
 - To reset counters
 - **To convert counters into per-second signals**
 - To delete old data
 - To group labels



MCQ

- What does sum by (handler) achieve in PromQL?
 - Deletes labels
 - Groups time series by handler
 - Filters errors only
 - Creates new metrics



Answer

- What does sum by (handler) achieve in PromQL?
 - Deletes labels
 - **Groups time series by handler**
 - Filters errors only
 - Creates new metrics



MCQ

- Why are histograms used for latency?
 - They reduce storage cost
 - They store log messages
 - They track distributions of values
 - They replace gauges



Answer

- Why are histograms used for latency?
 - They reduce storage cost
 - They store log messages
 - **They track distributions of values**
 - They replace gauges



MCQ

- Why are percentiles preferred over averages for latency?
 - They are easier to calculate
 - They hide slow requests
 - They show tail latency
 - They reduce cardinality



Answer

- Why are percentiles preferred over averages for latency?
 - They are easier to calculate
 - They hide slow requests
 - **They show tail latency**
 - They reduce cardinality



Kubernetes Monitoring Environment

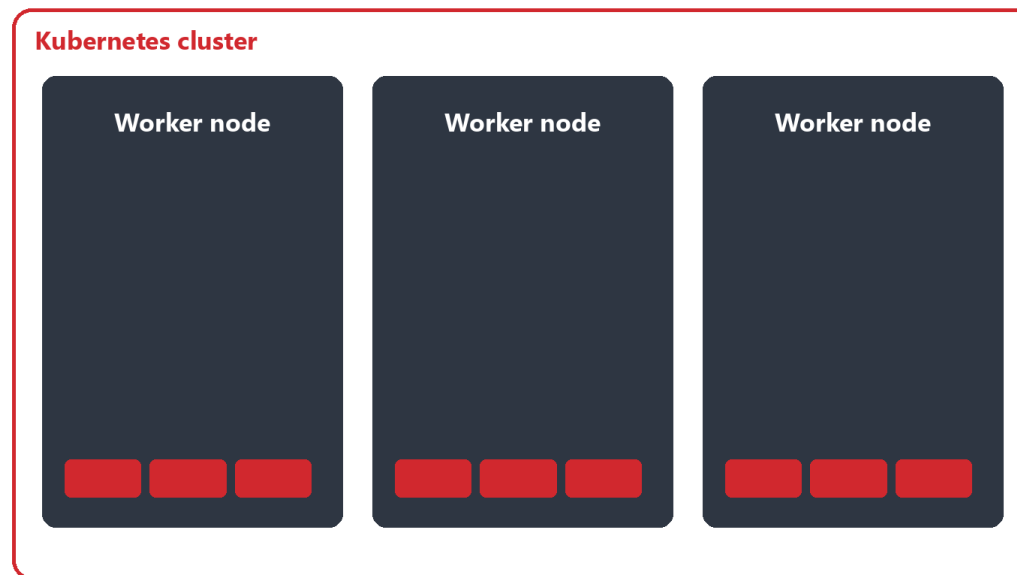
- This lab drops you into a real Kubernetes cluster running production-style workloads, not a toy demo. The point is to get comfortable reading a full monitoring stack the way you would on a real platform team.
 - EKS cluster
 - Multiple nodes
 - Real workloads
 - Production patterns



The Cluster Mental Model

- A cluster is just a pool of machines running your workloads, with a control plane scheduling pods onto worker nodes. Monitoring has to cover both layers, hosts and apps, because either can be the source of a problem.
 - Control plane
 - Worker nodes
 - Pods and services
 - Shared network

■ The cluster mental model



Monitor nodes and apps

Why We Monitor the Cluster

- Clusters never sit still: pods restart, nodes fill up, and traffic shifts constantly. Without monitoring you are blind to all of it, and you cannot tell a healthy reschedule from the start of an outage.
 - Node health
 - Pod health
 - Resource usage
 - Service behavior

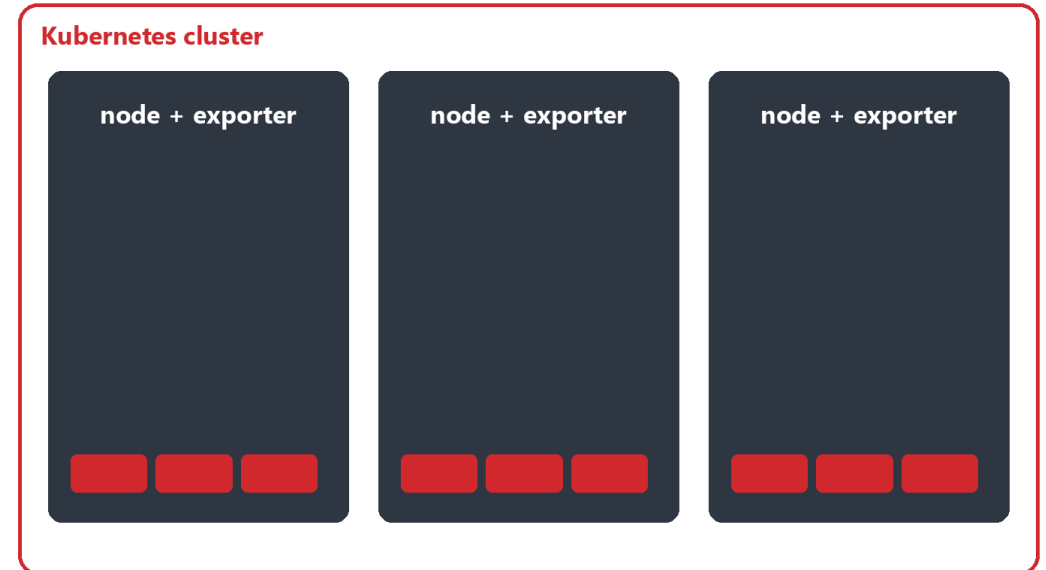
What we watch

-  Node health
-  Pod health
-  Resource usage
-  Service behavior

Node Exporter DaemonSet

- Node Exporter is the agent that reports the health of each machine itself. Running one copy on every node, it exposes CPU, memory, disk, and network so you can see when a host, not the app, is the problem.
 - CPU usage
 - Memory usage
 - Disk usage
 - Network usage

■ Node Exporter on every node

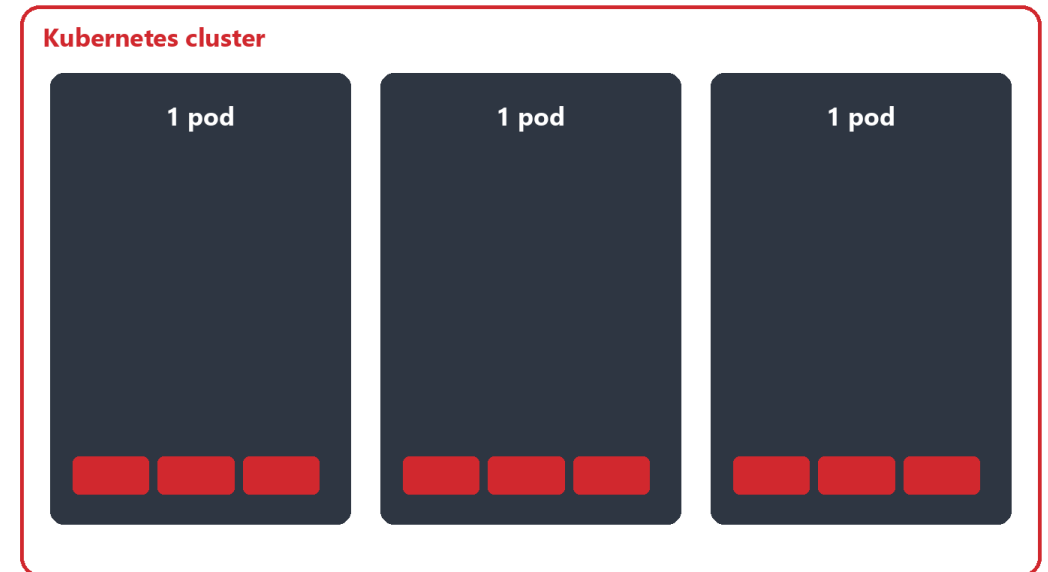


CPU, memory, disk, network

What a DaemonSet Means

- A DaemonSet is the Kubernetes way of saying run exactly one of these on every node. That guarantee is what gives Node Exporter complete coverage, even as nodes are added or replaced.
 - One per node
 - Scales with nodes
 - Infrastructure visibility
 - Cluster wide data

■ **DaemonSet: one pod per node**

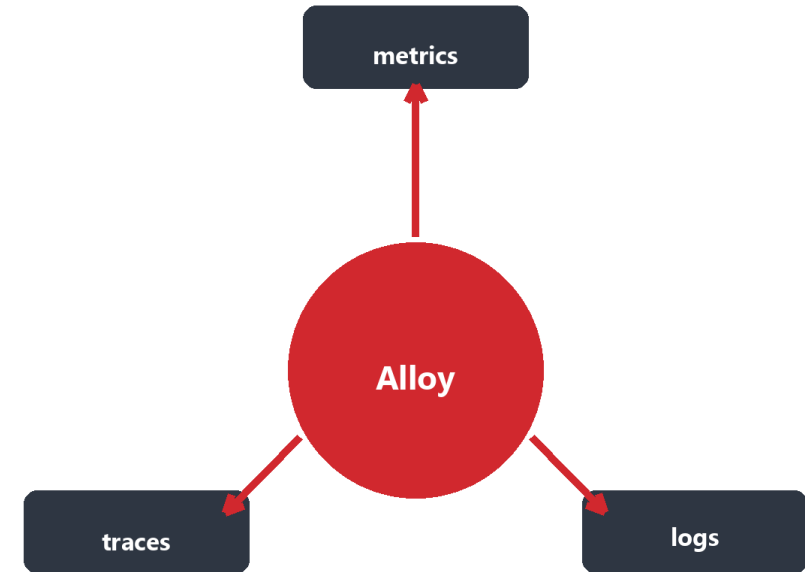


Guarantees full coverage

Grafana Alloy Agent

- Grafana Alloy is the collection agent that lives inside the cluster. It scrapes metrics, gathers logs, and forwards traces, acting as the single pipeline that feeds your observability backends.
 - Scrapes metrics
 - Collects logs
 - Forwards traces
 - Acts as a pipeline

■ Grafana Alloy agent

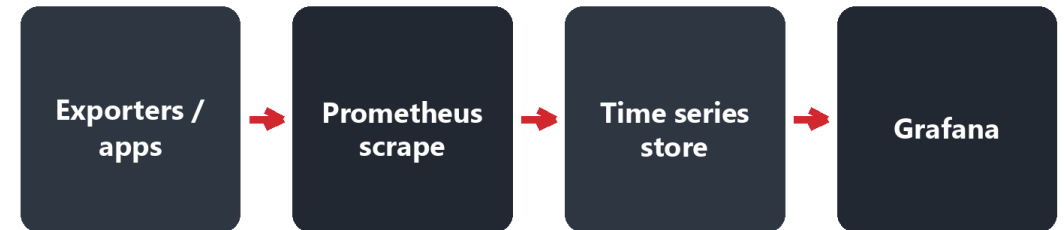


One pipeline for telemetry

Metrics Flow in the Cluster

- Metrics take a clear path: exporters and apps expose them, Prometheus scrapes them on a schedule, and the series land in storage ready to query. Knowing that path tells you where to look when a metric goes missing.
 - Node exporter
 - Service metrics
 - Prometheus scraping
 - Time series storage

■ Metrics flow in the cluster



Logs Flow in the Cluster

- Logs follow their own path out of the cluster. Each pod writes to stdout, an agent like Alloy or Promtail collects it, and Loki stores it, giving you one searchable place for every service's logs.
 - Pod logs
 - Alloy or Promtail
 - Loki storage
 - Searchable logs

■ Logs flow in the cluster



Grafana as the Control Center

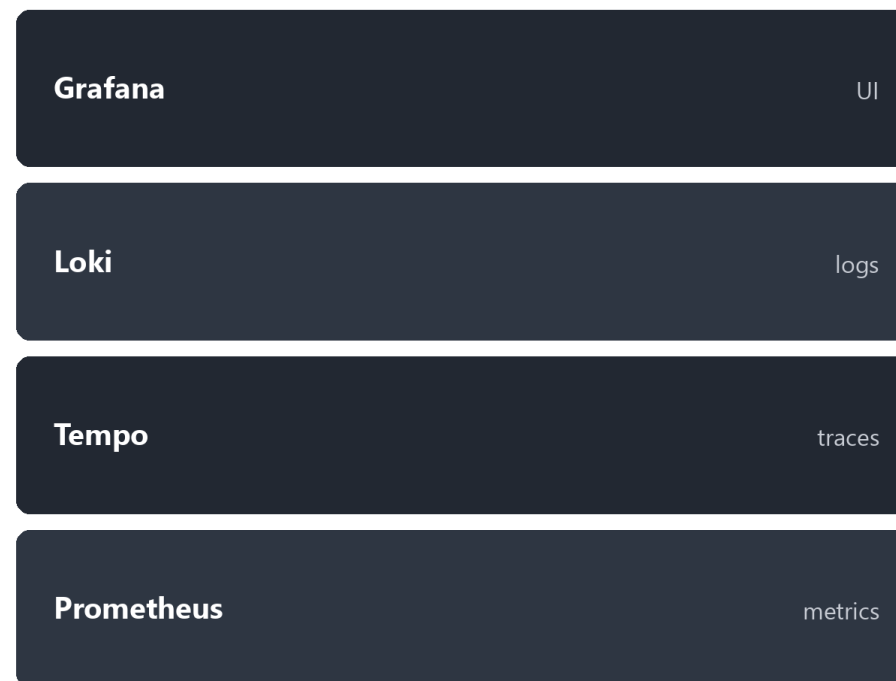
- Grafana is where humans actually meet all this telemetry. Dashboards, Explore, and alerts sit on top of the metrics, logs, and traces, so the team works from one screen instead of four tools.
 - Dashboards
 - Explore mode
 - Alerts
 - Single pane of glass



The LGTM Stack

- The LGTM stack is how all of this fits together, with each tool owning one signal. Loki for logs, Grafana for the UI, Tempo for traces, and Prometheus for metrics means no failure mode is invisible.
 - Loki for logs
 - Grafana for UI
 - Tempo for traces
 - Prometheus for metrics

■ The LGTM stack



Cluster Telemetry Sources

- Signals in Kubernetes come from every layer, not just your code. Nodes, the Kubernetes components themselves, your applications, and system services all emit telemetry, and a complete picture needs all of them.
 - Nodes
 - Kubernetes components
 - Applications
 - System services

Cluster telemetry sources

Nodes

K8s components

Applications

System services

Infrastructure vs Application Signals

- It helps to separate two kinds of signal. Infrastructure metrics like CPU and pod scheduling tell you about platform health; application metrics like request and error rates tell you about user experience.
 - CPU and memory
 - Pod scheduling
 - Request rates
 - Error rates

Infrastructure vs application

Infrastructure

- CPU and memory
- Pod scheduling
- platform health

Application

- Request rates
- Error rates
- user experience

Why Agents Exist

- Agents run inside the environment so you do not have to instrument every machine by hand. They discover targets automatically, collect in a standard way, and scale with the cluster, which makes monitoring hundreds of pods practical.
 - Automatic discovery
 - Standard collection
 - Low overhead
 - Scalable

Why agents exist

- ✓ Automatic discovery
- ✓ Standard collection
- ✓ Low overhead
- ✓ Scales with the cluster

Service Monitors

- A ServiceMonitor is how you tell Prometheus what to scrape, declaratively. Instead of editing a central config, you describe which services expose metrics and on what port, and the Prometheus Operator does the rest.
 - Target selection
 - Scrape endpoints
 - Label mapping
 - Dynamic discovery

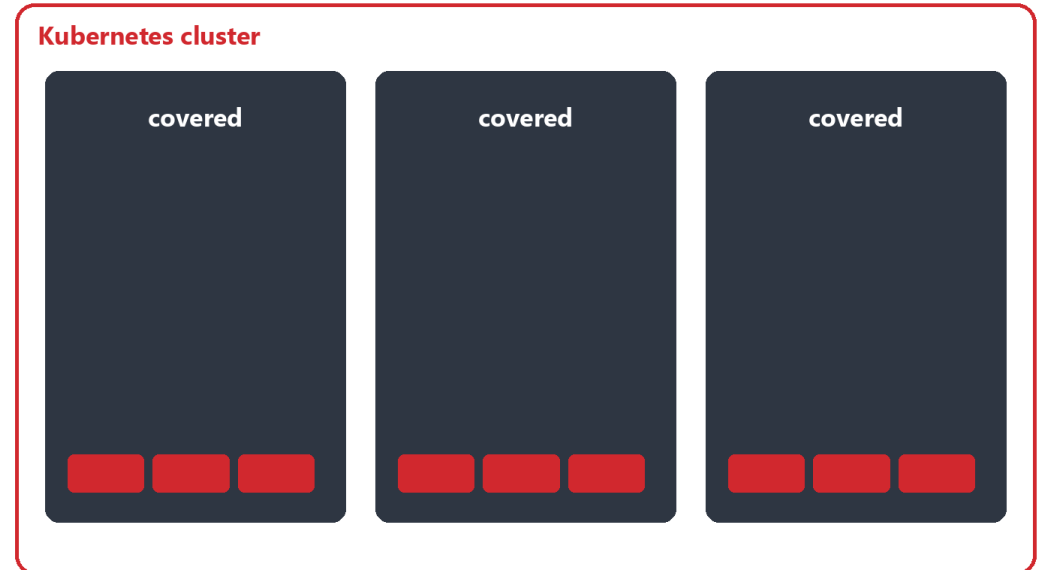
ServiceMonitor: what to scrape

```
kind: ServiceMonitor
selector: app=checkout
endpoints: port=metrics
```


Cluster Wide Visibility

- Once exporters and agents are deployed, every node and service reports in. The payoff is a real-time picture with no blind spots, so an incident does not hide in the one component nobody was watching.
 - No blind spots
 - Unified telemetry
 - Consistent metrics
 - Full coverage

Cluster-wide visibility



No blind spots

Observability as a Platform

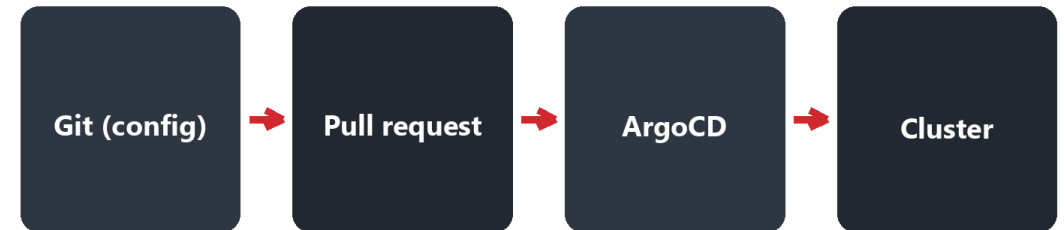
- When monitoring is built into the cluster, it stops being a tool each team bolts on and becomes a shared service. New workloads are observable by default, which is what separates a platform from a pile of scripts.
 - Always on
 - Shared service
 - Built into cluster
 - Operational foundation



Why GitOps for Monitoring

- Putting monitoring config in Git makes the whole stack reproducible and reviewable. A dashboard or alert rule changes through a pull request, so you get history, rollbacks, and the same setup in every environment.
 - Version control
 - Change history
 - Rollbacks
 - Consistency

■ Why GitOps for monitoring



ArgoCD Mental Model

- ArgoCD continuously compares what Git says should be running against what actually is, and fixes the difference. If someone hand-edits a dashboard in the cluster, ArgoCD notices the drift and restores the version in Git.
 - Declarative
 - Self healing
 - Automatic sync
 - Drift detection

■ ArgoCD keeps cluster in sync

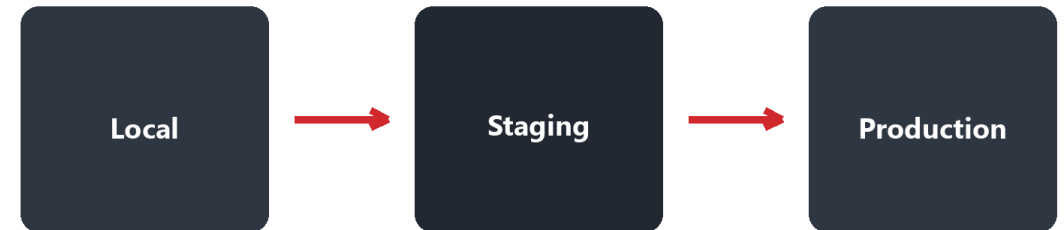


Detects and fixes drift

From Dev to Production

- The same monitoring patterns travel from your laptop to production unchanged. Only the scale and complexity grow, which means what you learn on a local cluster applies directly to the real thing.
 - Local clusters
 - Staging
 - Production
 - Same stack

■ Same stack everywhere



The Goal of This Lab

- The aim here is understanding the environment, not memorizing commands. If you leave thinking in terms of signals, sources, and a single pane of glass, the specific tools will make sense wherever you work next.
 - System thinking
 - Platform awareness
 - Observability mindset
 - Confidence in the stack

What you take away

- ✓ System thinking
- ✓ Platform awareness
- ✓ Observability mindset
- ✓ Confidence in the stack